

MADRE

Agentic
System

• MODEL-AGNOSTIC • DELAYED • REASONING • EFFORT • [Beta v0.1]

Runtime Domain Model

Scvria Software

June 20, 2026

Contents

Runtime Domain Model	1
1.1 Design Rule	1
1.2 Minimal Canonical Flow	1
1.3 Conventions	2
1.3.1 Type Conventions	2
1.3.2 Process Conventions	2
1.3.3 Class Inventory	3
1.4 Accepted UML Registry	4
1.5 Structural Relations	4
1.5.1 Runtime Ownership	4
1.5.2 Agent Family	5
1.5.3 Planning And Invocation	6
1.5.4 Context, Material, Knowledge, And Learning	7
1.5.5 Trace And Authority Evidence	7
1.6 Class Definitions	8
1.6.1 MADREKernel	8
1.6.2 ReasoningModule	9
1.6.3 MADREAgent	9
1.6.4 SimpleAgent	10
1.6.5 TwinAgent	10
1.6.6 ComplexAgent	10
1.6.7 SystemAgent	11
1.6.8 Workflow	11
1.6.9 AgentRoutine	11
1.6.10 AgentAction	12
1.6.11 ModelAction	12
1.6.12 ModelBinding	13
1.6.13 ReasoningRequest	13
1.6.14 ReasoningResponse	14
1.6.15 ReasoningPlan	14
1.6.16 ReasoningTask	15
1.6.17 AgentRequest	15
1.6.18 AgentResponse	16
1.6.19 ContextBundle	16
1.6.20 ReasoningArtifact	17
1.6.21 KnowledgeRecord	17
1.6.22 KnowledgeCandidate	18

1.6.23	LearningCandidate	18
1.6.24	PolicyDecision	19
1.6.25	RuntimeJournal	19
1.6.26	RuntimeEvent	20
1.7	Validation Checklist	20

Runtime Domain Model

MADRE principles are rooted in the domain architecture rather than brute computation. The runtime class map below is intentionally conservative: a class appears only when it owns stable identity, lifecycle, ownership boundary, responsibility, canonical-flow participation, or boundary participation that cannot safely be collapsed into another object.

This chapter incorporates the accepted classmap baseline. It removes storage adapters, inference records, inference outputs, generic boundary-contract containers, and conversational record classes from the runtime domain map. Those concepts remain possible infrastructure, product wording, retained material, or event payloads, but they are not accepted runtime classes here.

1.1 Design Rule

A class exists in this map only when it has stable runtime identity, stable ownership boundary, stable lifecycle, stable responsibility that cannot be expressed as a value inside another class, required participation in the canonical **MADRE** flow, or boundary participation needed for valid composition. Attributes preserve identity, ownership, lifecycle, relation, traceability, or accepted invariants. Operations appear only when caller, preconditions, postconditions, and side effects are clear enough at architecture level.

1.2 Minimal Canonical Flow

1. An access surface submits a user, system, module, or agent-originated reasoning need to the kernel. The surface is not authority.
2. **MADREKernel** resolves an explicit valid target module when present; otherwise it falls back to the Core Module role. The kernel does not semantically choose the best module.
3. A **ReasoningModule** receives a **ReasoningRequest** and answers, clarifies, blocks, delegates, creates a plan, or fails inside its own boundary.
4. Structured or delayed work becomes a **ReasoningPlan** containing **ReasoningTask** occurrences.
5. A **Workflow** may be applied by module-owned planning to generate or refine tasks. It is a planning-time blueprint, not an executable action.
6. Execution-time tasks invoke a module-owned **MADREAgent**, **AgentRoutine**, **AgentAction**, or **ModelAction** through a concrete **AgentRequest**.
7. **ModelAction** uses an allowed **ModelBinding**; generated material returns through **AgentResponse** and, when materialized, **ReasoningArtifact**.

8. `KnowledgeRecord`, `KnowledgeCandidate`, and `LearningCandidate` remain separate lifecycle concepts. Produced material does not become knowledge or learning by itself.
9. `PolicyDecision` records boundary outcomes. `RuntimeEvent` records runtime evidence. `RuntimeJournal` appends the local trace.

1.3 Conventions

1.3.1 Type Conventions

- **UUID:** Stable identifier
- **Text:** Human-readable or structured textual description
- **Instant:** Timestamp
- **Ref<T>:** Stable reference to another runtime object
- **List<T>:** Ordered collection
- **Set<T>:** Unordered collection without duplicates
- **Map<K,V>:** Key-value structure
- **Optional<T>:** Absent or present value

1.3.2 Process Conventions

Term	Meaning
<code>minimizeContext</code>	Context adapted to the target scope, with sensitive or unrelated material omitted.
Boundary composition	Construction of valid planning or execution paths from declared module, agent, workflow, routine, action, model, context, storage, privacy, and learning constraints.
Planning-time compatibility	Compatibility used when <code>Workflow</code> creates or refines <code>ReasoningTask</code> objects.
Execution-time compatibility	Compatibility required before <code>AgentRoutine</code> , <code>AgentAction</code> , or <code>ModelAction</code> invocation.
State transition	A guarded process recorded through <code>RuntimeEvent</code> , not a plain enum assignment.

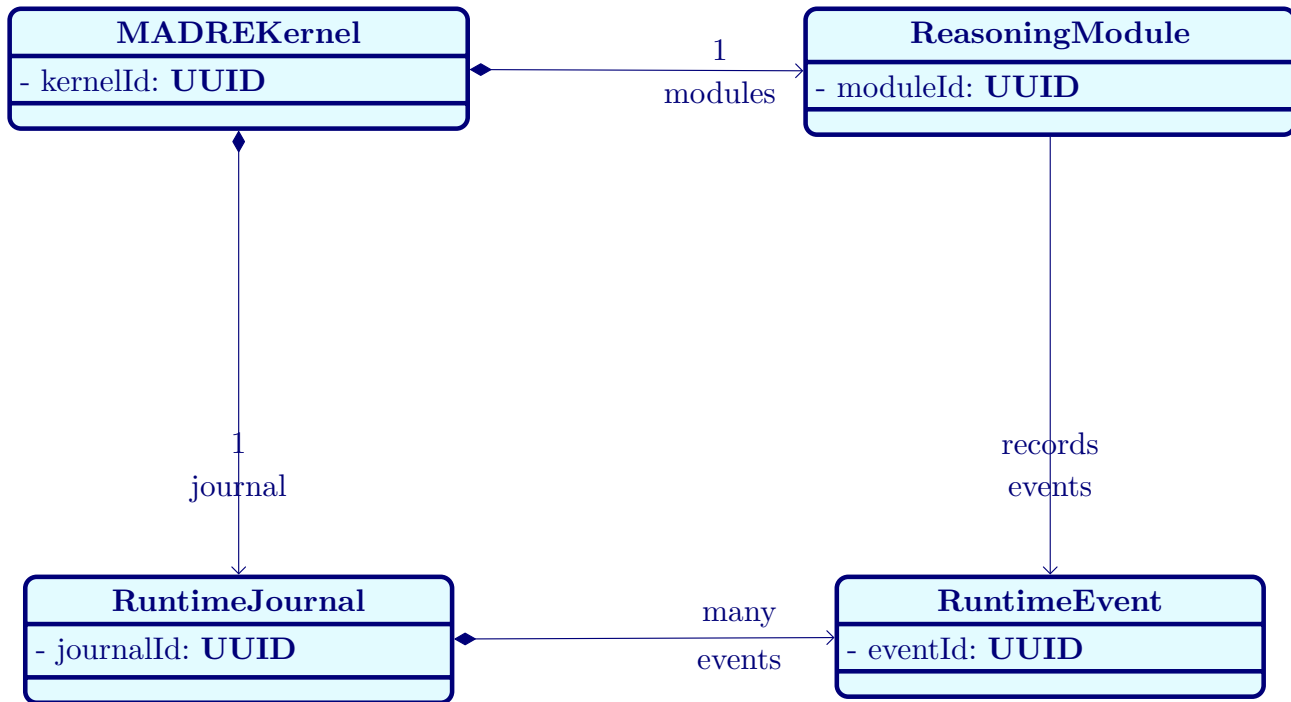
1.3.3 Class Inventory

Accepted classes	Open runtime areas
MADREKernel, ReasoningModule	Scheduler internals, storage ports, access-surface adapters, device-resource bindings.
MADREAgent, SimpleAgent, TwinAgent, ComplexAgent, SystemAgent	Concrete lane scheduling, review implementation, live-interaction adapter methods.
Workflow, AgentRoutine, AgentAction, ModelAction, ModelBinding	Capability metadata taxonomy, model backend adapter details, resource profiles, action subtype catalogues.
ReasoningRequest, ReasoningResponse, ReasoningPlan, ReasoningTask, AgentRequest, AgentResponse	Request status taxonomy, task transition operations, dependency resolution, quality-gate internals.
ContextBundle, KnowledgeRecord, LearningCandidate, ReasoningArtifact, KnowledgeCandidate	Truth taxonomy, lifecycle taxonomy, artifact subtype catalogue, promotion and rollback mechanics.
PolicyDecision, RuntimeEvent, RuntimeJournal	Event taxonomy, policy-rule engine internals, query/index implementation, evidence payload schema.

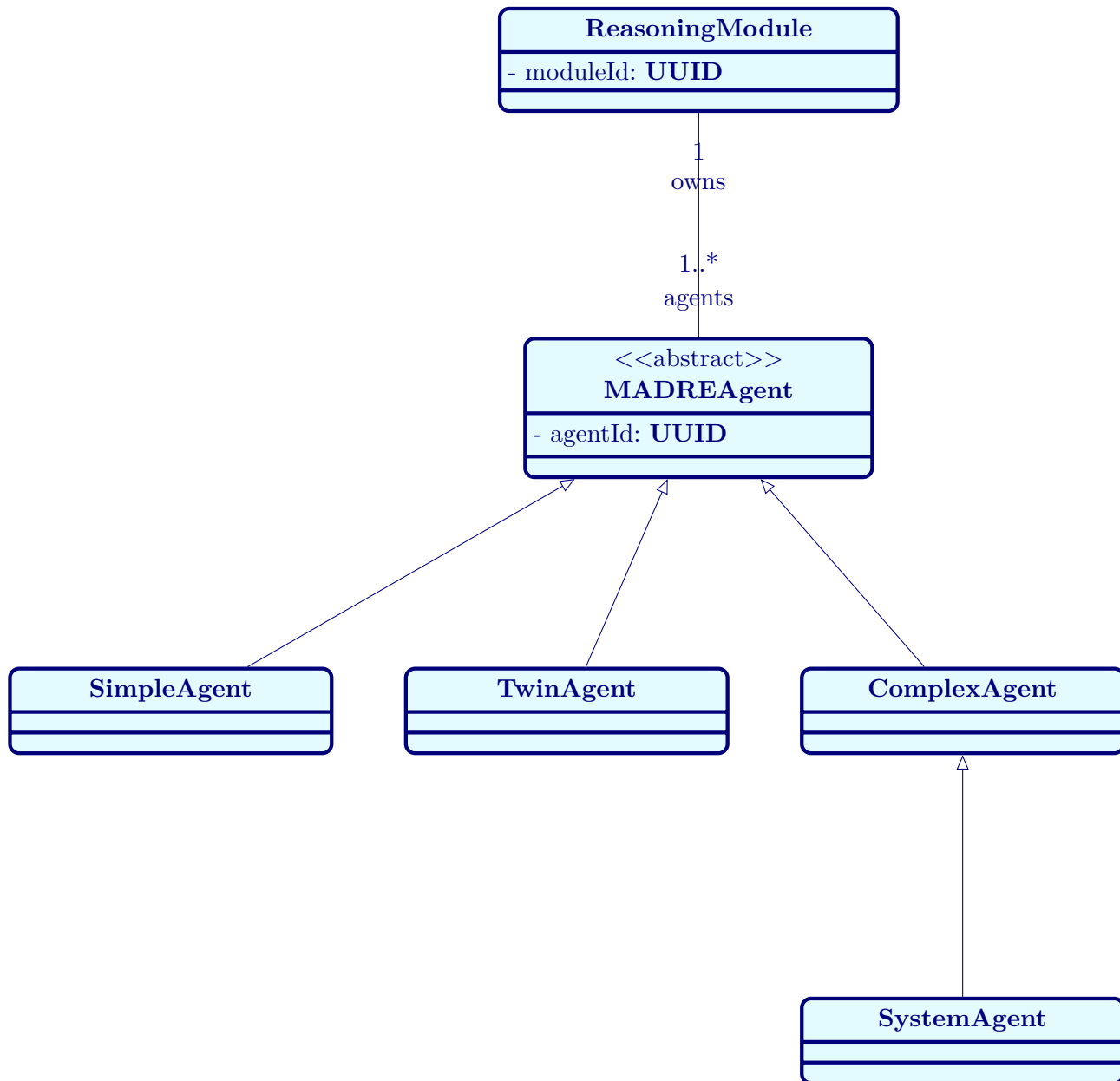
1.4 Accepted UML Registry

1.5 Structural Relations

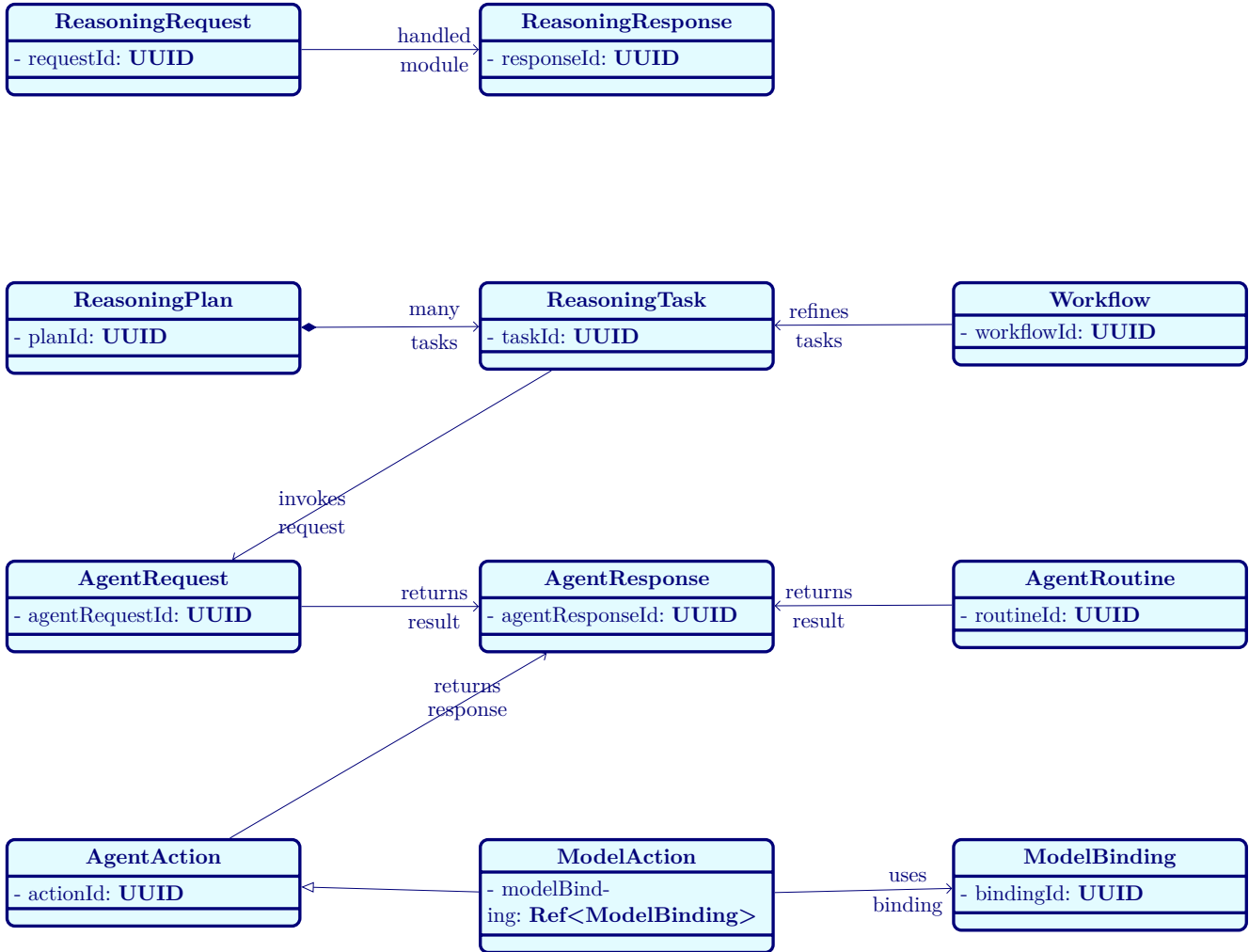
1.5.1 Runtime Ownership



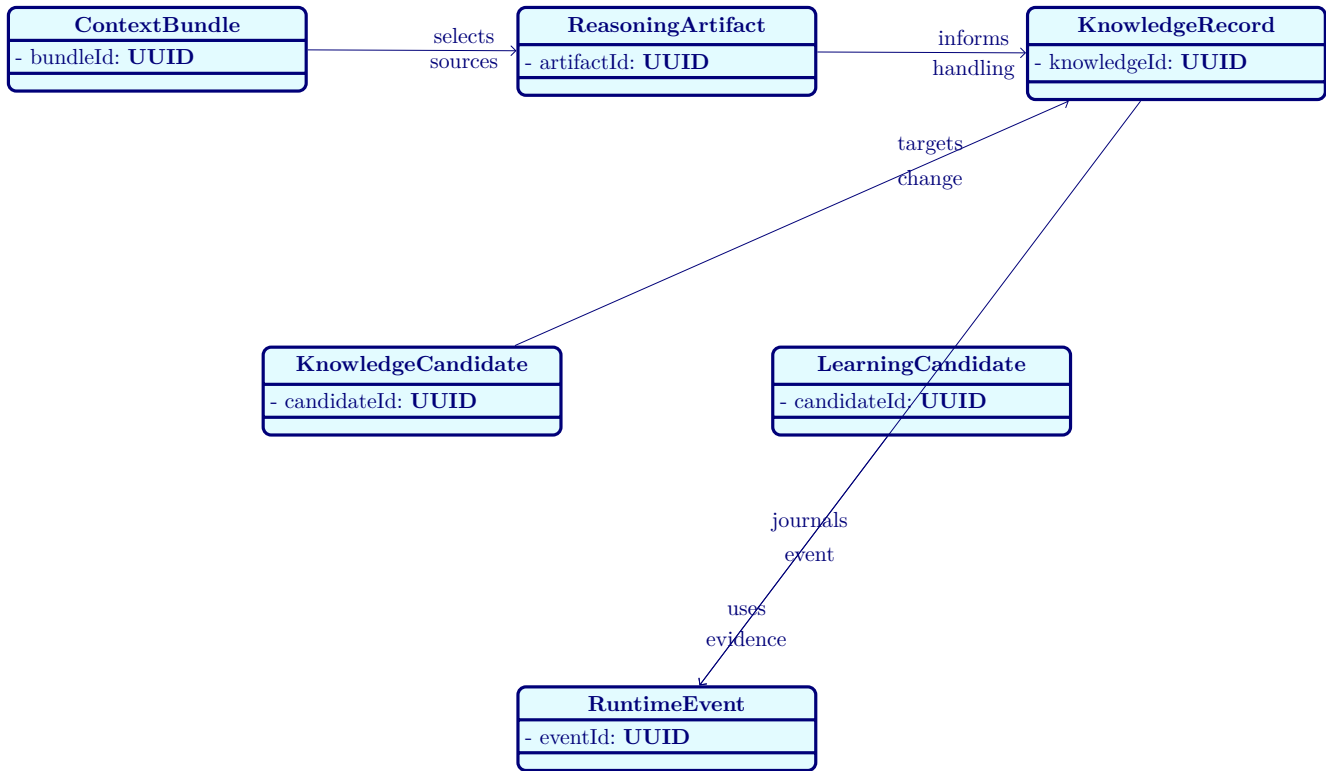
1.5.2 Agent Family



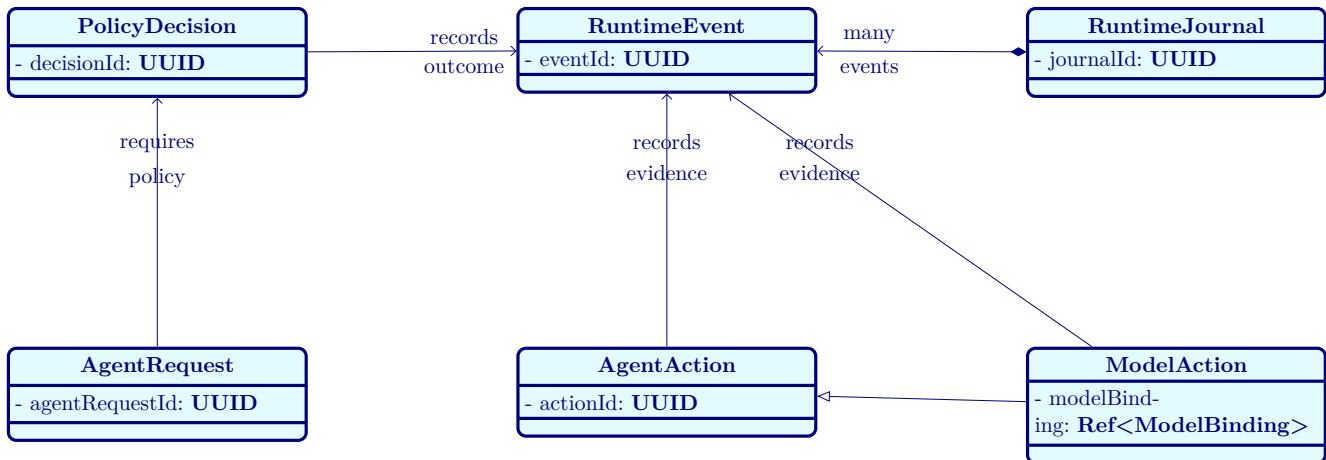
1.5.3 Planning And Invocation



1.5.4 Context, Material, Knowledge, And Learning



1.5.5 Trace And Authority Evidence



1.6 Class Definitions

1.6.1 MADREKernel

MADRE Agentic System

Class Definition
MADREKernel

MADREKernel: MADREKernel coordinates valid targets and runtime trace without becoming the reasoning actor.

- kernelId: UUID

Attributes:

- coreModule: Ref<ReasoningModule>

- moduleRegistry: Set<Ref<ReasoningModule>>

- runtimeJournal: RuntimeJournal

Operations: -NoValue-

Notes:

Deterministic local runtime coordinator.

Registers modules, assigns the Core Module role, resolves explicit module targets, falls back to the Core Module when a target is missing or invalid, schedules delayed work, provides scoped runtime services, and appends events.

It does not reason, semantically route, inspect module-private knowledge, execute actions, invoke models, promote knowledge, or learn.

Storage assignments, scheduler internals, access-surface adapters, and hardware bindings are implementation-defined infrastructure.

1.6.2 ReasoningModule

MADRE Agentic System

Class Definition
ReasoningModule

ReasoningModule: ReasoningModule owns semantic reasoning and local governance inside a module boundary.

Attributes:

- moduleId: UUID
- mainAgent: Ref<SystemAgent>
- agents: Set<Ref<MADREAgent>>
- workflows: Set<Workflow>
- actions: Set<AgentAction>

Operations: -NoValue-

Notes:

Governed reasoning and data-governance unit for a bounded domain, topic, user scope, system scope, or operational scope.

Owns module knowledge, local context governance, allowed agents, workflows, routines, actions, model bindings, learning boundaries, and risk limits.

Handles requests by answering, clarifying, blocking, delegating, creating a plan, or failing.

It does not become the kernel, directly access another module's private data, or widen the authority of its agents and actions.

1.6.3 MADREAgent

MADRE Agentic System

Class Definition
MADREAgent

<AbstractClass>

MADREAgent: MADREAgent executes module-authorized invocations.

Attributes: - agentId: UUID

Operations: -NoValue-

Notes:

Abstract module-owned runtime actor that executes concrete **AgentRequest** objects under module boundaries.

Uses provided context and runtime-constructed execution context, invokes only allowed routines or actions, and returns **AgentResponse** material.

It does not own kernel resources, widen its own authority, or promote produced material into knowledge or learning.

1.6.4 SimpleAgent

MADRE Agentic System

Class Definition
SimpleAgent

SimpleAgent: SimpleAgent directly executes valid requests.

Attributes:

Notes:

Concrete MADREAgent for direct execution where no mandatory review or complex lane behavior is required. It returns one AgentResponse and does not gain reflection, review, or learning authority.

1.6.5 TwinAgent

MADRE Agentic System

Class Definition
TwinAgent

TwinAgent: TwinAgent is justified by the checked-output postcondition.

Attributes:

Notes:

Concrete MADREAgent whose invariant is response production plus mandatory review before final dispatch. The review path does not widen action authority and unchecked material cannot satisfy a task that requires checked output.

1.6.6 ComplexAgent

MADRE Agentic System

Class Definition
ComplexAgent

ComplexAgent: ComplexAgent coordinates deeper behavior without gaining extra authority.

Attributes:

Notes:

Concrete MADREAgent capable of foreground continuity, deeper reasoning supervision, and reflective/evaluative work. Foreground, reasoning, and reflection lanes are behavioral capacities; scheduling and threading are not class attributes. Reflection may propose LearningCandidate objects but cannot activate learning.

1.6.7 SystemAgent

MADRE Agentic System

Class Definition
SystemAgent

SystemAgent: SystemAgent is the module-bound complex agent used for ordinary module operation.

Attributes:

Notes:

Module-bound **ComplexAgent** that serves as the module's main operational agent.
 Its distinguishing property is strong local binding to one **ReasoningModule**, not global authority.
 The Main Agent is a role held by one module-owned **SystemAgent**; it is not a separate class.

1.6.8 Workflow

MADRE Agentic System

Class Definition
Workflow

Workflow: Workflow is reusable module planning structure.

- workflowId: UUID

Attributes: - ownerModule: Ref<ReasoningModule>
 - purpose: Text

Notes:

Module-level planning blueprint that generates or refines **ReasoningTask** occurrences inside a **ReasoningPlan**.
 It is not an actor, not an action, not an agent routine, and not a scheduled object by itself.
 Compatibility is planning-time compatibility under accumulated request constraints.

1.6.9 AgentRoutine

MADRE Agentic System

Class Definition
AgentRoutine

AgentRoutine: AgentRoutine is compound executable agent behavior.

- routineId: UUID

Attributes: - ownerAgent: Ref<MADREAgent>
 - purpose: Text

Notes:

Agent-level compound executable capability.
 It receives **AgentRequest**, may coordinate several lower-level steps or actions, and returns **AgentResponse**.
 It is execution-time capability, not a module workflow and not a replacement for **ReasoningPlan**.

1.6.10 AgentAction

MADRE Agentic System

Class Definition
AgentAction

AgentAction: AgentAction is the smallest bounded executable capability.

- actionId: UUID

Attributes: - ownerModule: Ref<ReasoningModule>
 - purpose: Text

Operations: -NoValue-

Notes:

Smallest executable capability available under module or agent authority.

Executes deterministic, model-backed, integration-backed, or other bounded behavior only through a valid invocation path.

It does not authorize itself, execute from raw user text, or bypass runtime trace.

1.6.11 ModelAction

MADRE Agentic System

Class Definition
ModelAction

ModelAction: ModelAction is model-backed action execution under binding constraints.

Attributes: - modelBinding: Ref<ModelBinding>

Notes:

Specialized **AgentAction** that uses a model binding as part of execution.

It prepares model-backed input from **AgentRequest** and governed context, uses only allowed **ModelBinding**, and returns produced material through ordinary response or artifact paths.

Runtime evidence for model-backed execution is recorded through **RuntimeEvent**; no separate output class is accepted here.

1.6.12 ModelBinding

MADRE Agentic System

Class Definition
ModelBinding

ModelBinding: ModelBinding constrains model-backed execution without becoming model authority.

Attributes:

- bindingId: UUID
- description: Text

Notes:

Configuration boundary that constrains which model-backed execution options a model-backed action or agent may use.

Preserves model substitutability, local-first or authorized-remote limits, reproducibility expectations, and resource discipline.

It is not the model, not a runtime actor, not authority, and does not execute inference.

1.6.13 ReasoningRequest

MADRE Agentic System

Class Definition
ReasoningRequest

ReasoningRequest: ReasoningRequest represents module-level reasoning or delegation need.

Attributes:

- requestId: UUID
- objective: Text
- originRef: Ref<?>
- targetModule: Optional<Ref<ReasoningModule>>
- contextRef: Optional<Ref<ContextBundle>>
- scope: Text
- constraints: Text

Notes:

Passive request object for a reasoning need. It may originate from a user interaction, system event, module, agent, plan, or delegated task.

Carries objective, origin, explicit target when known, scope, constraints, and context references.

It does not build plans, schedule itself, execute work, authorize actions, or carry module-private knowledge directly across delegation boundaries.

1.6.14 ReasoningResponse

MADRE Agentic System

Class Definition ReasoningResponse

ReasoningResponse: ReasoningResponse carries module-level handling output.

- responseId: UUID
- outcome: Text

Attributes:

- materialRef: Optional<Ref<?>>
- artifactRefs: List<Ref<ReasoningArtifact>>
- planRef: Optional<Ref<ReasoningPlan>>

Notes:

Passive module-level response to a ReasoningRequest.

Represents answer, clarification, block, delegation, plan-created, or failure material.

It does not execute work, authorize actions, or promote contained material into knowledge or learning.

1.6.15 ReasoningPlan

MADRE Agentic System

Class Definition ReasoningPlan

ReasoningPlan: ReasoningPlan owns delayed reasoning task structure.

- planId: UUID
- Attributes:**
- ownerModule: Ref<ReasoningModule>
 - sourceRequest: Ref<ReasoningRequest>
 - tasks: List<Ref<ReasoningTask>>

Notes:

Durable reasoning-continuity object for structured or delayed work.

Owns task occurrences and preserves enough plan structure for scheduling, inspection, dependency management, failure handling, and resumption.

It does not execute itself, allocate kernel resources, bypass module ownership, or promote results into knowledge or learning.

1.6.16 ReasoningTask

MADRE Agentic System

Class Definition
ReasoningTask

ReasoningTask: ReasoningTask records planned work before concrete invocation.

Attributes:

- taskId: UUID
- targetKind: Text
- targetRef: Optional<Ref<?>>
- dependencies: List<Ref<ReasoningTask>>
- contextRequirement: Text
- boundaryRequirement: Text

Notes:

Passive planned execution occurrence inside a ReasoningPlan.

Targets a delegated ReasoningRequest, an executable agent capability, an AgentRoutine, an AgentAction, a ModelAction, or a Workflow only when planning expansion or refinement is intended.

It does not contain a prebuilt AgentRequest before execution selection and does not execute itself.

1.6.17 AgentRequest

MADRE Agentic System

Class Definition
AgentRequest

AgentRequest: AgentRequest is the concrete invocation envelope.

Attributes:

- agentRequestId: UUID
- objective: Text
- contextRef: Ref<ContextBundle>
- targetAgent: Optional<Ref<MADREAgent>>
- targetCapability: Optional<Ref<?>>
- effectiveConstraints: Text

Notes:

Concrete executable invocation context constructed by module orchestration.

Carries only the minimized context and effective constraints needed for one valid agent, routine, action, or model-backed execution.

It does not replace ReasoningRequest, schedule work, or make context valid merely by containing it.

1.6.18 AgentResponse

MADRE Agentic System

Class Definition
AgentResponse

AgentResponse: AgentResponse wraps concrete execution output.

Attributes:

- agentResponseId: UUID
- outcome: Text
- artifactRefs: List<Ref<ReasoningArtifact>>
- eventRefs: List<Ref<RuntimeEvent>>

Notes:

Concrete invocation result returned by a MADREAgent, AgentRoutine, AgentAction, or ModelAction.
 Carries result material, artifact references, block material, refusal material, or failure material.
 It is not knowledge, policy, authority, learning, or routing by itself.

1.6.19 ContextBundle

MADRE Agentic System

Class Definition
ContextBundle

ContextBundle: ContextBundle carries minimized selected context without making it authoritative.

Attributes:

- bundleId: UUID
- ownerModule: Ref<ReasoningModule>
- sourceRefs: List<Ref<?>>
- scope: Text
- sensitivity: Text
- intendedUse: Text

Notes:

Governed context prepared for one request, task, agent invocation, routine invocation, action invocation, or model-backed execution.
 Carries selected material or references while preserving source, scope, sensitivity, purpose, minimization, and revocation semantics.
 Source material does not become instruction or authority by inclusion.

1.6.20 ReasoningArtifact

MADRE Agentic System
Class Definition
ReasoningArtifact

ReasoningArtifact: ReasoningArtifact preserves produced material without granting truth or authority.

Attributes:

- artifactId: UUID
- ownerModule: Ref<ReasoningModule>
- contentRef: Ref<?>
- originEventRef: Ref<RuntimeEvent>
- intendedUse: Text

Notes:

Materialized output or captured result produced by reasoning or action execution.

Represents produced material that may be returned, inspected, exported, referenced, corrected, discarded, or used as input to later handling.

It is not knowledge, truth, learning, policy, or action authority by default.

1.6.21 KnowledgeRecord

MADRE Agentic System
Class Definition
KnowledgeRecord

KnowledgeRecord: KnowledgeRecord is uncertain retained module knowledge.

Attributes:

- knowledgeId: UUID
- ownerModule: Ref<ReasoningModule>
- contentRef: Ref<?>
- provenanceRefs: List<Ref<?>>
- scope: Text
- sensitivity: Text
- intendedUse: Text
- truthLevel: Text
- truthAuthority: Text
- confidence: Text
- lifecycleState: Text

Notes:

Module-owned retained material that may later be used as known context, evidence, preference, style, fiction, procedure, hypothesis, correction, memory, or working belief under explicit metadata.

It is broad uncertain knowledge, not factual truth by default and not a subclass of ReasoningArtifact in this baseline.

It does not authorize actions, promote itself across module boundaries, or become learning by being stored.

1.6.22 KnowledgeCandidate

MADRE Agentic System

Class Definition KnowledgeCandidate

KnowledgeCandidate: KnowledgeCandidate quarantines pending knowledge-boundary change.

Attributes:

- candidateId: UUID
- ownerModule: Ref<ReasoningModule>
- proposedChange: Text
- sourceRefs: List<Ref<?>>
- targetKnowledgeRef: Optional<Ref<KnowledgeRecord>>
- status: Text

Notes:

Pending change to a module knowledge boundary.

Represents proposed inclusion, reclassification, correction, ownership transfer, trust-level change, intended-use change, promotion, detachment, invalidation, or related mutation.

It is not active knowledge, not learning, and does not authorize future use until resolved.

1.6.23 LearningCandidate

MADRE Agentic System

Class Definition LearningCandidate

LearningCandidate: LearningCandidate stores proposed behavior improvement without activating it.

Attributes:

- candidateId: UUID
- ownerModule: Ref<ReasoningModule>
- proposedImprovement: Text
- evidenceRefs: List<Ref<?>>
- scope: Text
- rollbackRefs: List<Ref<?>>
- status: Text

Notes:

Evaluated proposed improvement to module behavior, evaluation, routing, workflows, routines, context selection, trust calibration, model selection, or other module function.

Remains inactive until promoted through the applicable learning boundary.

Saving KnowledgeRecord material is not learning by itself.

1.6.24 PolicyDecision

MADRE Agentic System

Class Definition PolicyDecision

PolicyDecision: PolicyDecision records a boundary outcome without granting authority by itself.

Attributes:

- decisionId: UUID
- outcome: Text
- reason: Text
- affectedRef: Optional<Ref<?>>
- eventRef: Optional<Ref<RuntimeEvent>>

Notes:

Passive record of an explicit boundary or authorization outcome.

Records allow, block, or authorization-required outcomes and links them to affected action, data, autonomy, or boundary evidence when known.

It does not execute policy logic, authorize by itself, or override the composed boundary mechanism.

1.6.25 RuntimeJournal

MADRE Agentic System

Class Definition RuntimeJournal

RuntimeJournal: RuntimeJournal preserves append-only runtime history.

Attributes: - journalId: UUID

Operations: -NoValue-

Notes:

Append-only local runtime trace mechanism.

Appends **RuntimeEvent** entries and stores or references payloads needed to inspect request handling, plan lifecycle, task execution, context construction, model-backed action execution, action execution, produced material, policy outcomes, failures, candidates, correction, rollback, deletion, and recovery.

It is not a reasoning actor, knowledge repository, general logging framework, or authorization mechanism.

1.6.26 RuntimeEvent

MADRE Agentic System

Class Definition
RuntimeEvent

RuntimeEvent: RuntimeEvent records runtime evidence without authorizing it.

- eventId: UUID
- occurredAt: Instant
- eventType: Text

Attributes:

- actorRef: Optional<Ref<?>>
- affectedRef: Optional<Ref<?>>
- summary: Text
- payloadRef: Optional<Ref<?>>

Notes:

Append-only record of something relevant that happened in the runtime.

Records request receipt, target resolution, plan creation, scheduling, task transition, context construction, boundary outcome, action execution, model-backed execution, artifact creation, response production, failure, correction, rollback, invalidation, supersession, deletion, detachment, knowledge-candidate transition, or learning-candidate transition.

It does not authorize what happened, execute behavior, make material true, or replace domain objects.

1.7 Validation Checklist

1. The accepted class inventory contains exactly the 26 baseline runtime classes.
2. The kernel remains deterministic coordination and target resolution, not semantic reasoning.
3. Core Module and Main Agent remain roles, not separate classes.
4. Modules own reasoning, knowledge, agents, workflows, routines, actions, and local governance.
5. Workflow is planning-time task generation or refinement, not executable work.
6. AgentRoutine, AgentAction, and ModelAction are execution-time capabilities invoked through concrete AgentRequest.
7. ReasoningTask does not contain a prebuilt AgentRequest before execution selection.
8. Model-backed execution is ModelAction plus ModelBinding, ordinary produced material, and RuntimeEvent evidence.
9. Produced material does not become knowledge, truth, policy, action authority, routing, behavior, or learning by itself.
10. KnowledgeRecord, KnowledgeCandidate, and LearningCandidate preserve separate lifecycle semantics.

11. Boundary enforcement is not reduced to a loose policy flag or generic container.
12. Storage mechanisms remain implementation-defined infrastructure until a stable domain boundary is proven.